Assignment 2: Image Filtering and Convolutional Neural Networks

By: Julia Reinhart

1. **Introduction:** This assignment covers two main components of computer vision – image filtering and convolutional neural networks (CNN). Image filtering enhances certain features of the image using different types of matrices and mathematical operations. This can make different features stand out more such as horizontal edges vs. vertical edges, blurring, and textures. CNN networks build upon this foundation of image filtering by using deep learning and a neural network to distinguish separate image classes such as differentiating between an image of a cat and a dog. CNN's can learn image filtering independently using the training data set and multiple hidden layers in the neural network. The main objective of this project was to first familiarize ourselves with the foundation of image filtering and then apply those concepts by building a convolutional neural network that can classify images into ten distinct categories of images.
2. **Part 1 - Image Filtering:**

- **Convolution and Kernels:** The mathematical operation for image filtering is called *convolution* in which a certain type of matrix, known as a kernel, is used to emphasize certain features of the image. The dot product of the specified matrix is calculated against each subregion image pixels, and we multiply element wise and then add the results together. This output represents for a single dot product represents the final singular pixel in the output image. The purpose of convolution is that through this process is that distinct kernels can emphasize different features such as textures, edges, etc.

  For example, to apply convolution with the four kernels below,

**Horizontal Edge Kernel**

$$K_{\text{horizontal}} = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

---

**Vertical Edge Kernel**

$$K_{\text{vertical}} = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

### Diagonal (Top-Left → Bottom-Right)

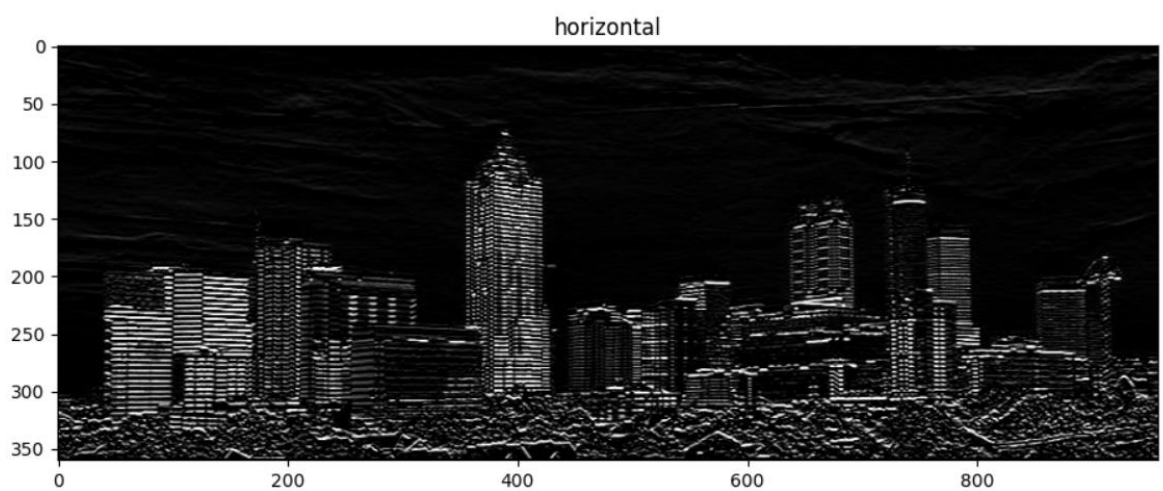
$$K_{\text{diag1}} = \begin{bmatrix} 0 & -1 & -1 \\ 1 & 0 & -1 \\ 1 & 1 & 0 \end{bmatrix}$$

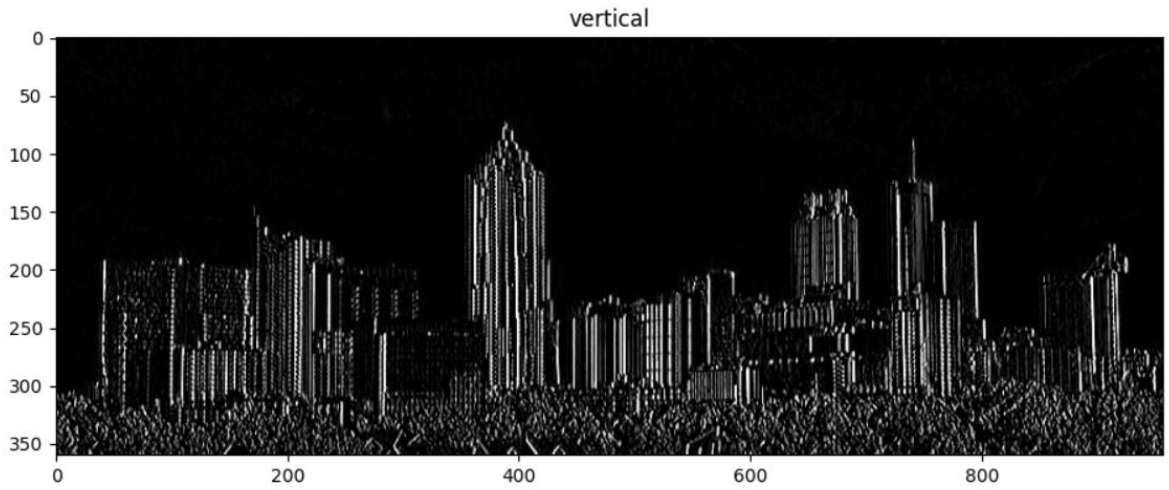
---

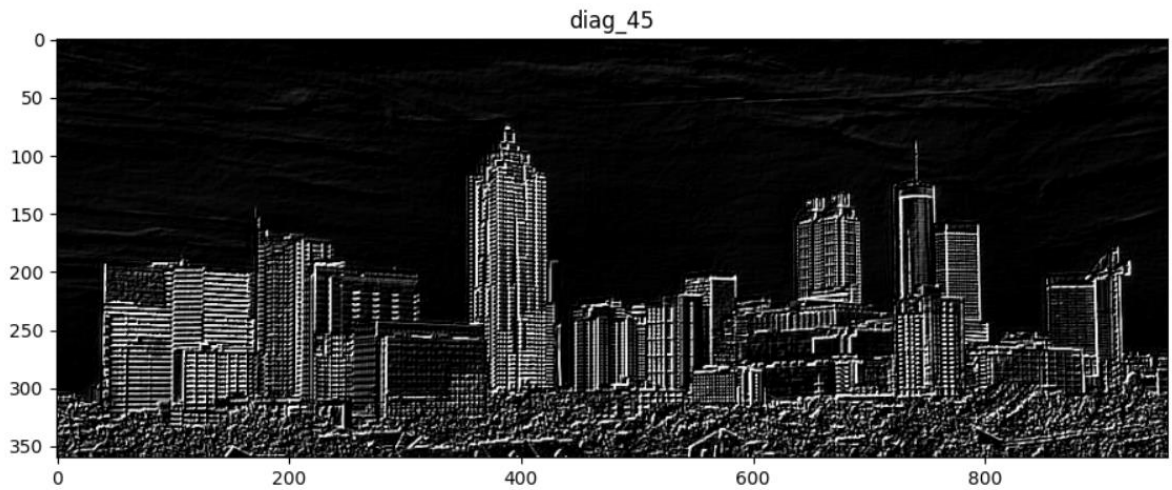
### Diagonal (Top-Right → Bottom-Left)

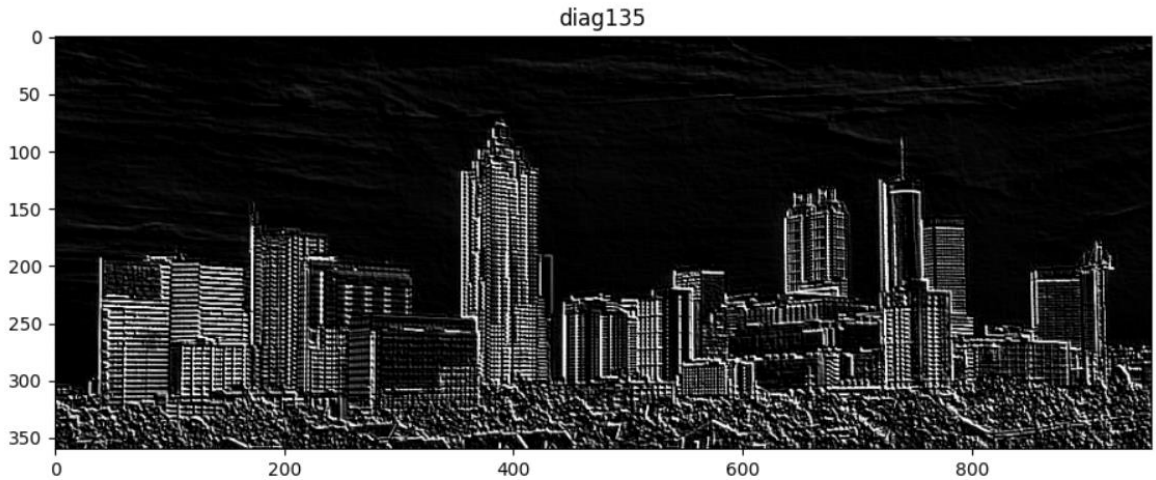
$$K_{\text{diag2}} = \begin{bmatrix} -1 & -1 & 0 \\ -1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$$

We will extract 4 different features, horizontal edges, vertical edges, and both types of diagonals. Below are the four different images we obtain from the four different types of convolutions:

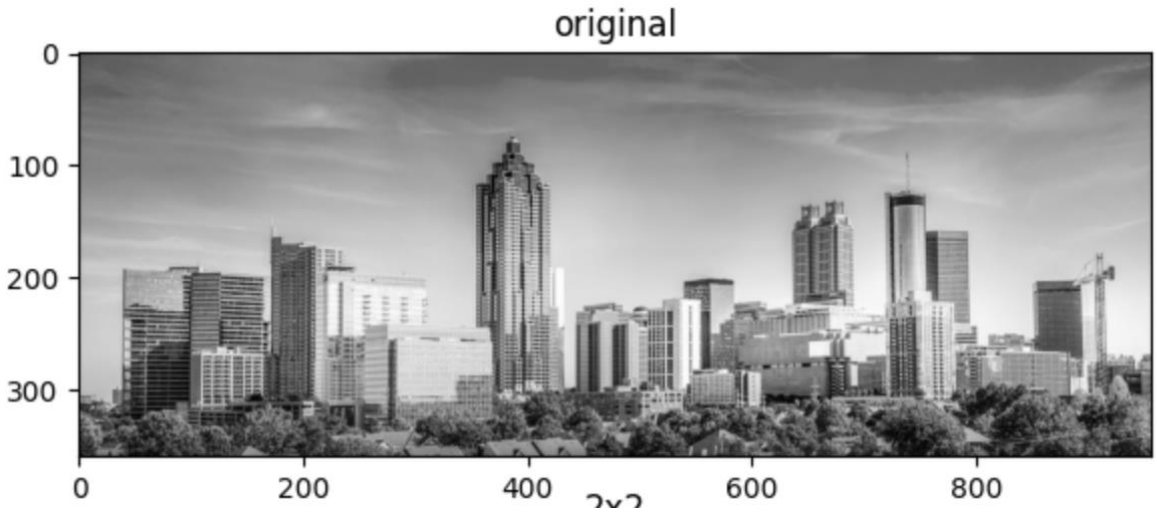


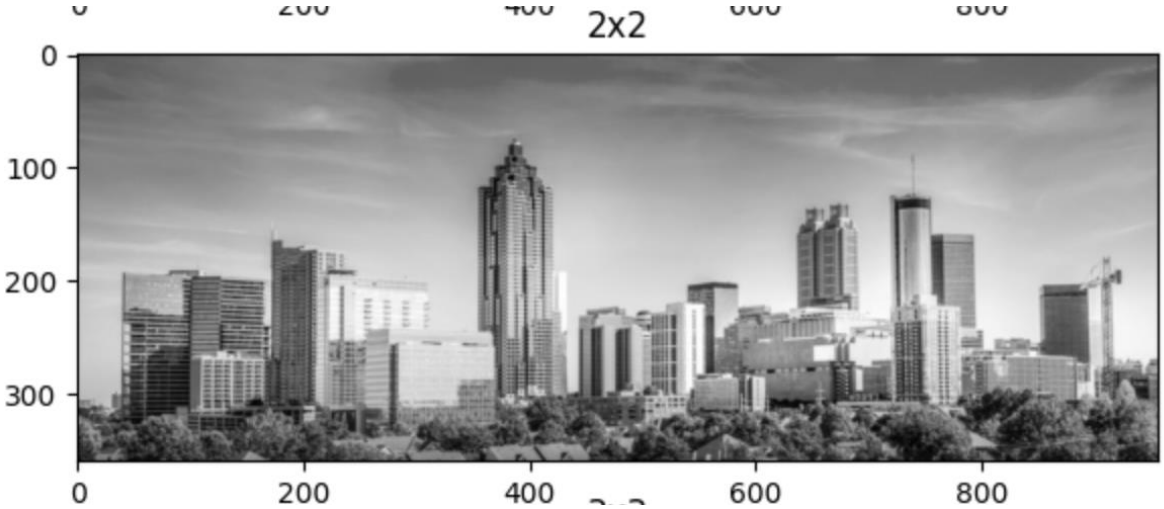

vertical
0
50
100
150
200
250
300
350
0
200
400
600
800

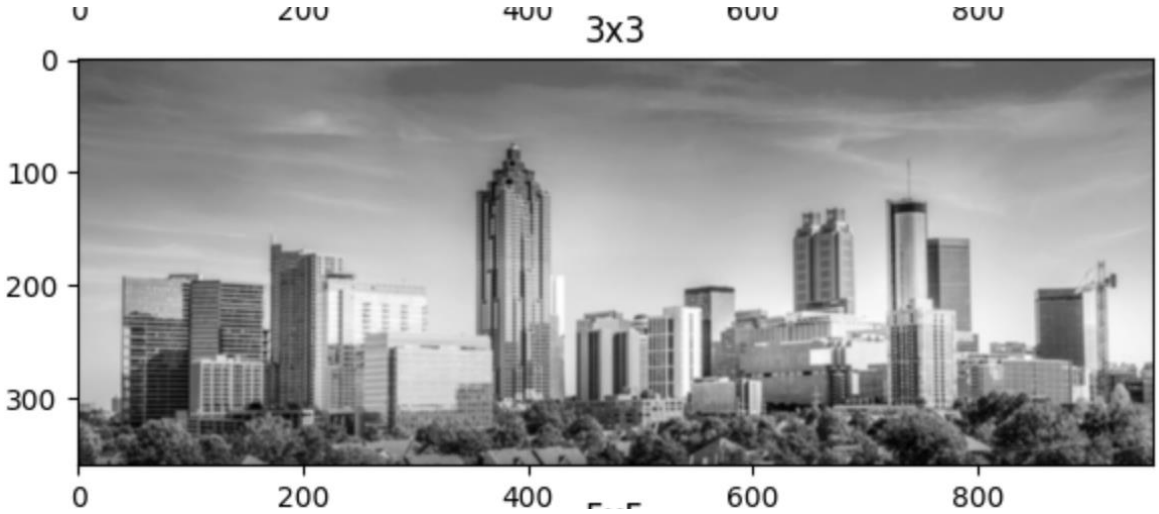

diag_45
0
50
100
150
200
250
300
350
0
200
400
600
800

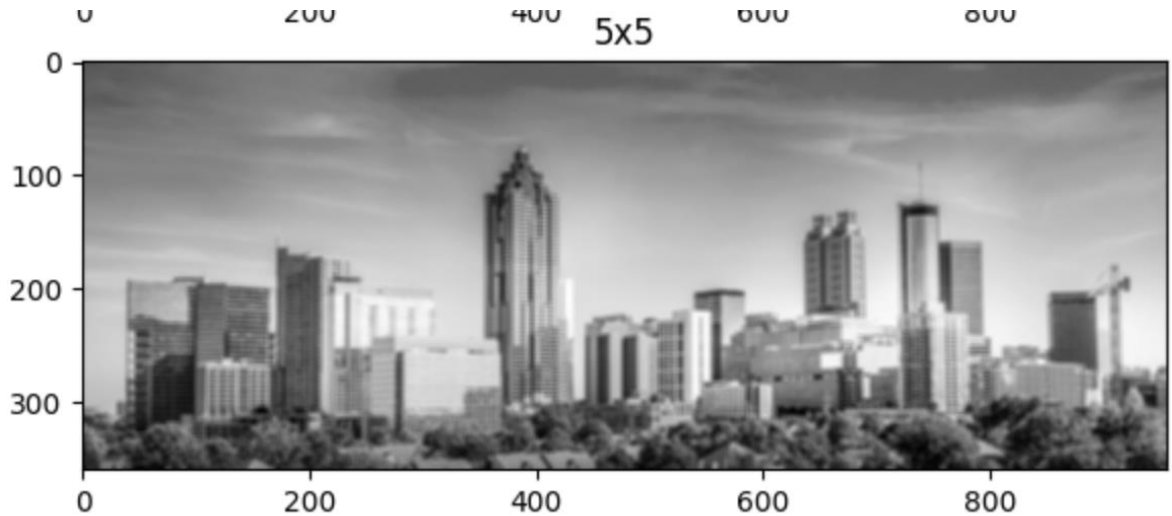



- **Image Blurring:** Blurring does a similar process to convolution. It picks a matrix of fixed size and sets each entry equal to 1's and normalizes the matrix by dividing by its magnitude. For example, for a matrix of size 3x3, we will divide each 1 by 9. Similarly, for a matrix with dimensions 2x2, we will divide each entry in the matrix by 4. Then, a dot product is repeated for each subset of pixels the same size as the kernel. The bigger the matrix, the bigger the average neighboring of pixels, and so the blurrier the result becomes. Below are the results from our notebook after blurring was implemented using matrix sizes of 2x2, 3x3, and 5x5 respectively contrasted against the original image.




2x2
0
100
200
300
0
200
400
600
800


3x3
0
100
200
300
0
200
400
600
800




Clearly, our results support this concept that the larger the matrix used in the blurring operation, the blurrier the resulting image becomes.

- **Edge Detection using the Sobel Operator:** To improve upon our past edge detection, the Sobel operator was used. The Sobel operator involves using a matrix similar to the horizontal and vertical matrices demonstrated previously, but it adds weights to the middle edges and results in a clearer edge depiction. Below are the two matrices used for the Sobel operator to emphasize horizontal and vertical edges respectively.

### Sobel X (detects vertical edges)

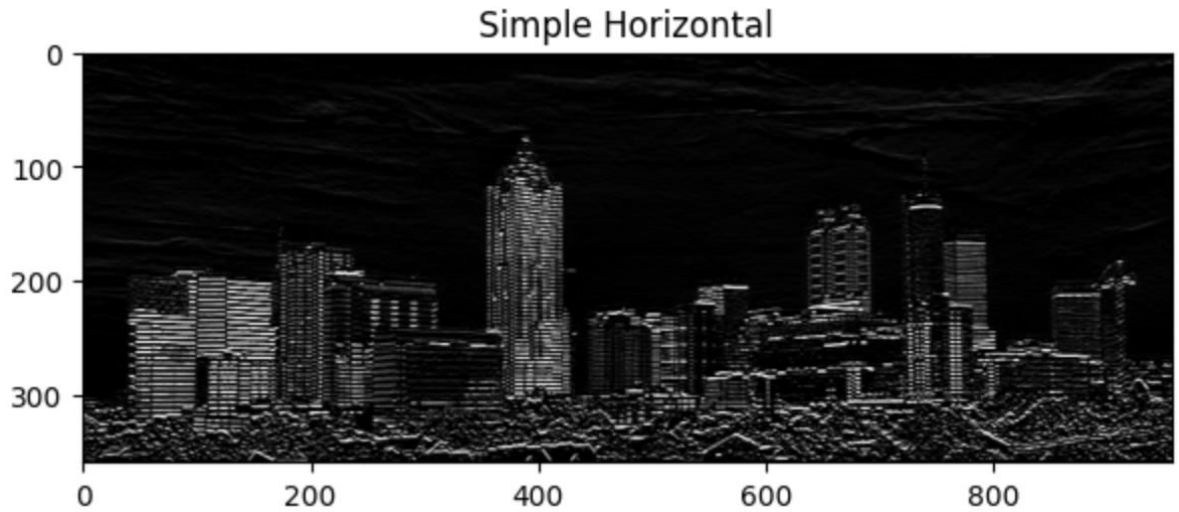
$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

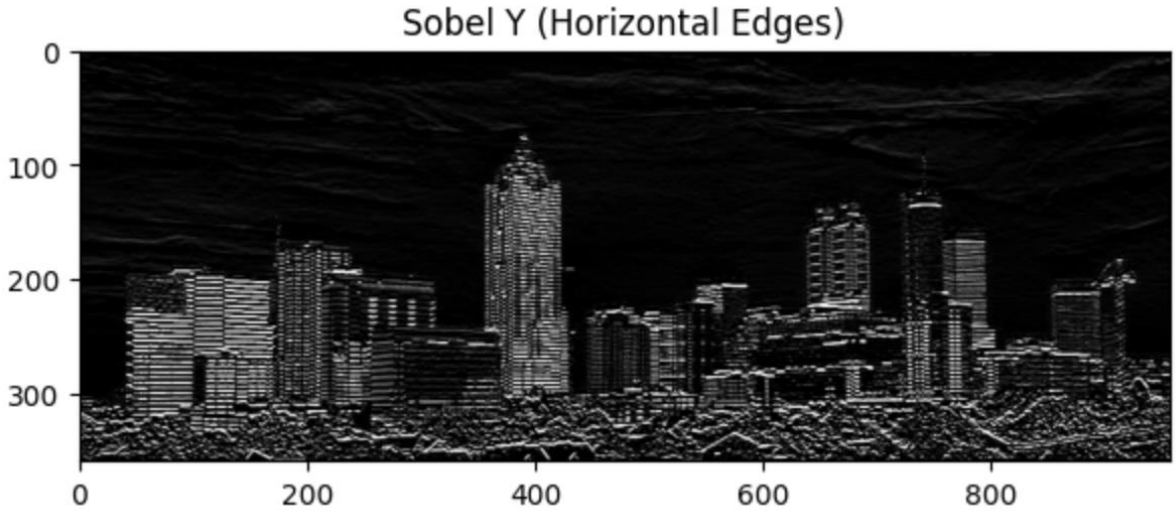
---

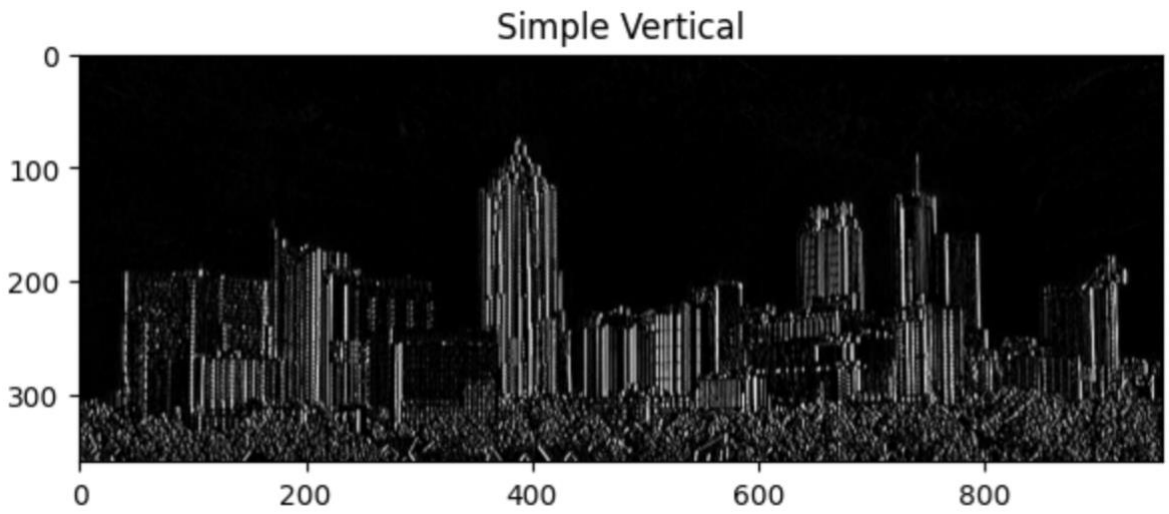
### Sobel Y (detects horizontal edges)

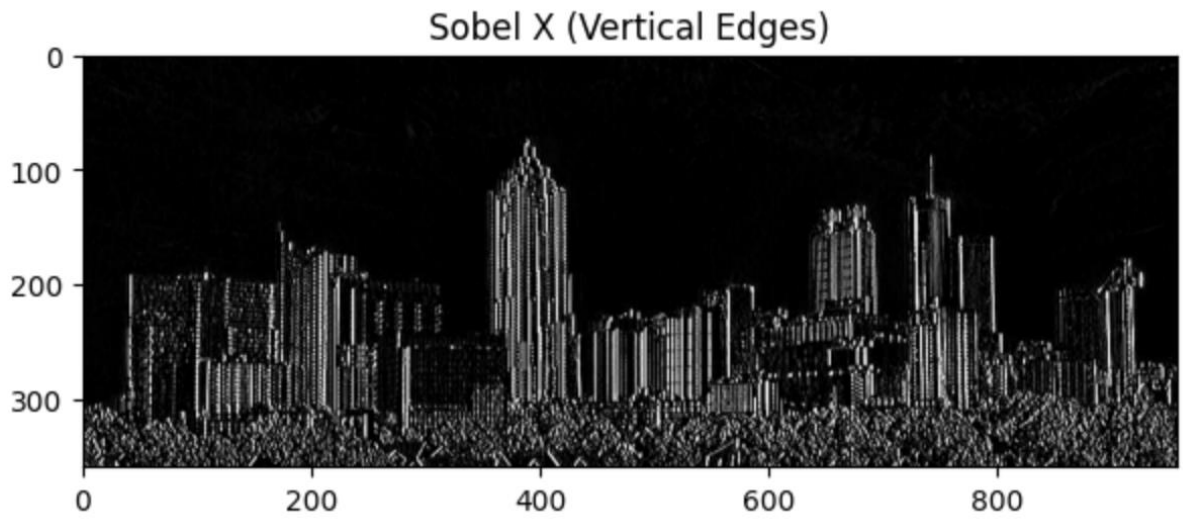
$$S_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

We now compare the output from simple horizontal and vertical edges with the Sobel horizontal and vertical edges. Below are the outputs from each:



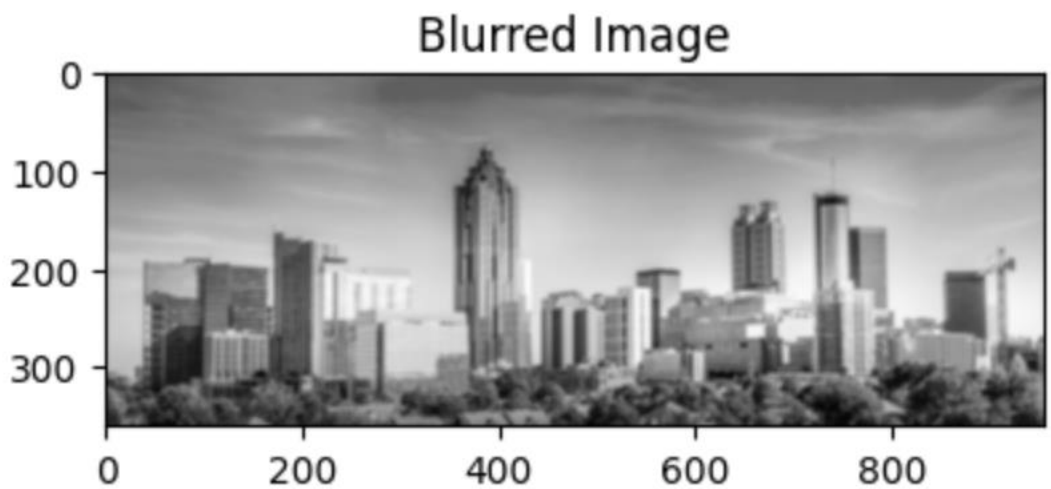



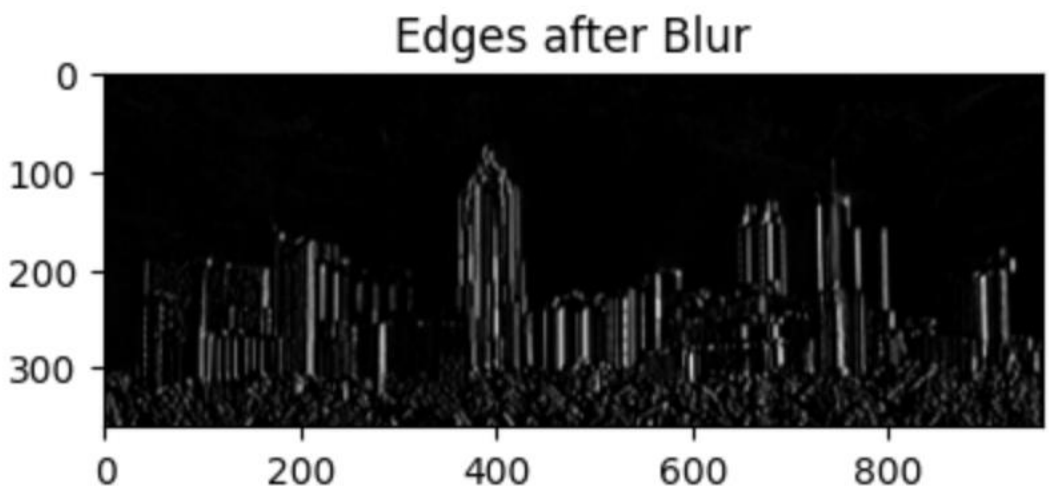



Clearly, from these images, we can see that the Sobel operator produces a clearer, smoother, more precise result.

- **Blur followed by Edge Detection:** Finally, a sequence of filters was applied in which we first, blurred the image, and subsequently, detected simple edges as usual. The findings were that since the blurring reduced the noise, the edges had a higher contrast and were more pronounced. Examine the images below where we illustrate both steps of (1) blurring the image and (2) computing traditional edges.




Overall, preprocessing an image such as smoothing out the noise by using the blur tactic, can drastically improve the quality of the final result.

3. **Part 2 - Convolutional Neural Network:**

- **Data:** The dataset used for our convolutional neural network was the CIFAR-10 dataset and downloads automatically with running the specified commands in the README.md document in the GitHub Repository. Using the given data, the objective of our model was to correctly classify images into one of the following ten categories: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck.
- **Architecture of Model:** The model built was configured to have three convolutional layers to increase accuracy and better track more robust features from the dataset. Below is the diagram depicting our convolutional neural network:

Input Image: $3 \times 32 \times 32$

↓

After Convolutional Layer 1: $32 \times 32 \times 32$ (using 32 filters total)

↓

ReLU (to introduce non-linearity)

↓

After MaxPool(2×2): $32 \times 16 \times 16$

↓

After Convolutional Layer 2: $64 \times 16 \times 16$ (using 64 filters)

↓

ReLU (to introduce non-linearity)

↓

After MaxPool (2×2): $64 \times 8 \times 8$

↓

Conv Layer 3: $128 \times 8 \times 8$ (using 128 filters)

↓

ReLU (to introduce non-linearity)

↓

After MaxPool (2×2): $128 \times 4 \times 4$

↓

Flatten

↓

Results in single dimensional vector of length 128 × 4 × 4 multiplied = 2048

↓

Fully Connected Layer (compresses 2048 total values into an output of 256 values)

↓

ReLU (to introduce non-linearity)

↓

Dropout (randomly turn off 25% of neurons during training to prevent overfitting)

↓

Fully Connected Layer (compresses the previous computed layer into 10 total classes)

↓

Final Output: Scores per Class

Note that a padding value of 1 was used in between each convolutional layer to keep dimensions fixed aside from the factor we are increasing or decreasing by using the MaxPool function.

Now, we explain the model step by step. First, we begin our model with an input image of dimensions 3x32x32. CIFAR-10 images always begin with dimensions of 32x32 pixels with 3 color channels, so our beginning dimensions are 3x32x32.

Next, we apply convolutional layer 1 using the code:

```
self.conv1 = nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3, padding=1)
```

Since we are upgrading from 3 to 32 filters, we now have the dimensions become 32x32x32 after convolutional layer 1. We have chosen a kernal size of 3 which translates into kernal dimensions of 3x3. This applies filters to each image using a 3x3 kernal similar to how the image filtering notebook worked, but this is learned automatically from the model. The padding size of 1 means the dimensions stay fixed and are only modified by the MaxPool function and not convolution.

Similarly, we apply convolutional layers 2 and 3 using the code continued onto the next page:

```
self.conv2 = nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3, padding=1)

self.conv3 = nn.Conv2d(in_channels=64, out_channels=128, kernel_size=3, padding=1)
```

The ReLU function and a MaxPool(2,2) function is applied between each layer shown by the following code:

```
x = self.pool(F.relu(self.conv1(x))) # 32x32 → 16x16

x = self.pool(F.relu(self.conv2(x))) # 16x16 → 8x8

x = self.pool(F.relu(self.conv3(x))) # 8x8 → 4x4

self.pool = nn.MaxPool2d(2, 2)
```

As a result, these lines of code explain the dimensions going from 32x32x32 to 32x16x16, 64x8x8, and 128x4x4 after each convolution layer respectively.

In this fashion, increasing the number of filters from 32 to 64 to 128 allows the network to learn more abstract, nuanced features. On the other hand, max pooling reduces the computation cost and noise allowing the model to concentrate on the most important information.

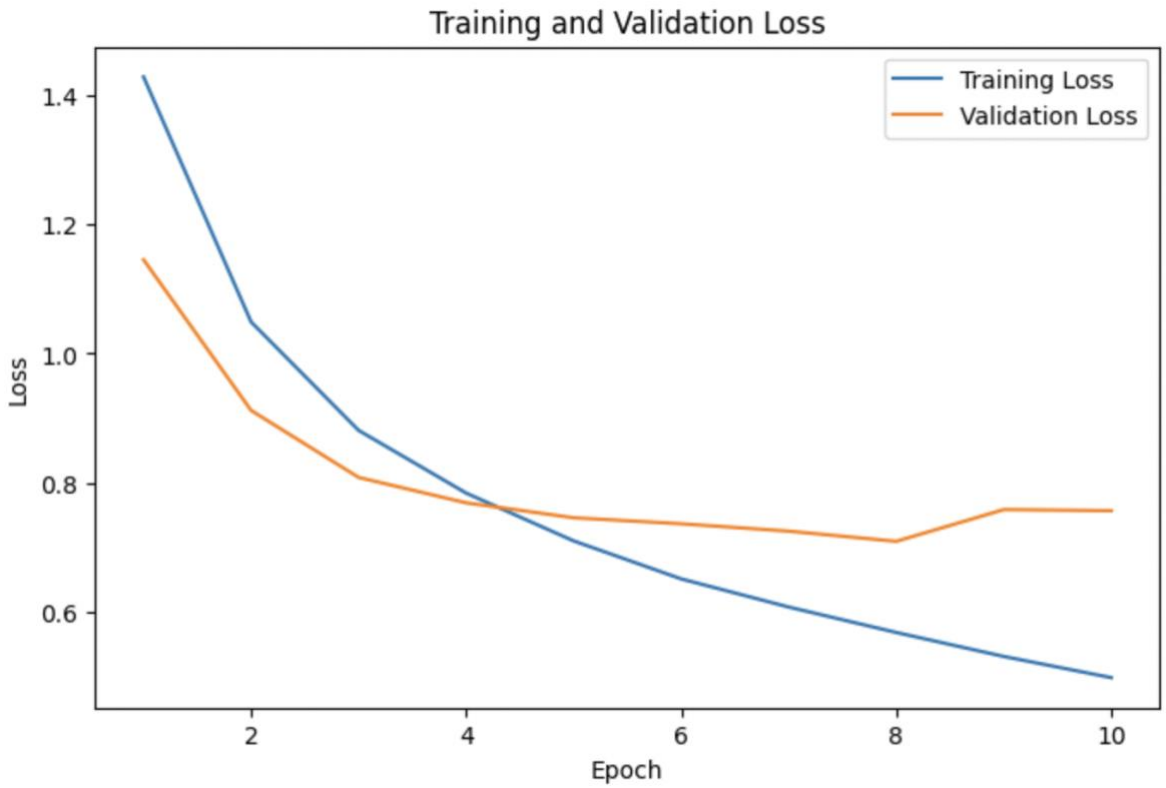
For reference, convolutional layers distinguish edges and features as shown in Part 1. ReLU activation addes more depth to the model by incorporating non-linearity, and Max pooling limits noise in the model by reducing the spatial size and emphasizing important features. Flattening transforms the 3D data by multiplying each dimension into a single output vector of one dimension. Next, layers become fully connected and handle the classification tasks, and using dropout reduces overfitting by randomly omitting neurons from the model.

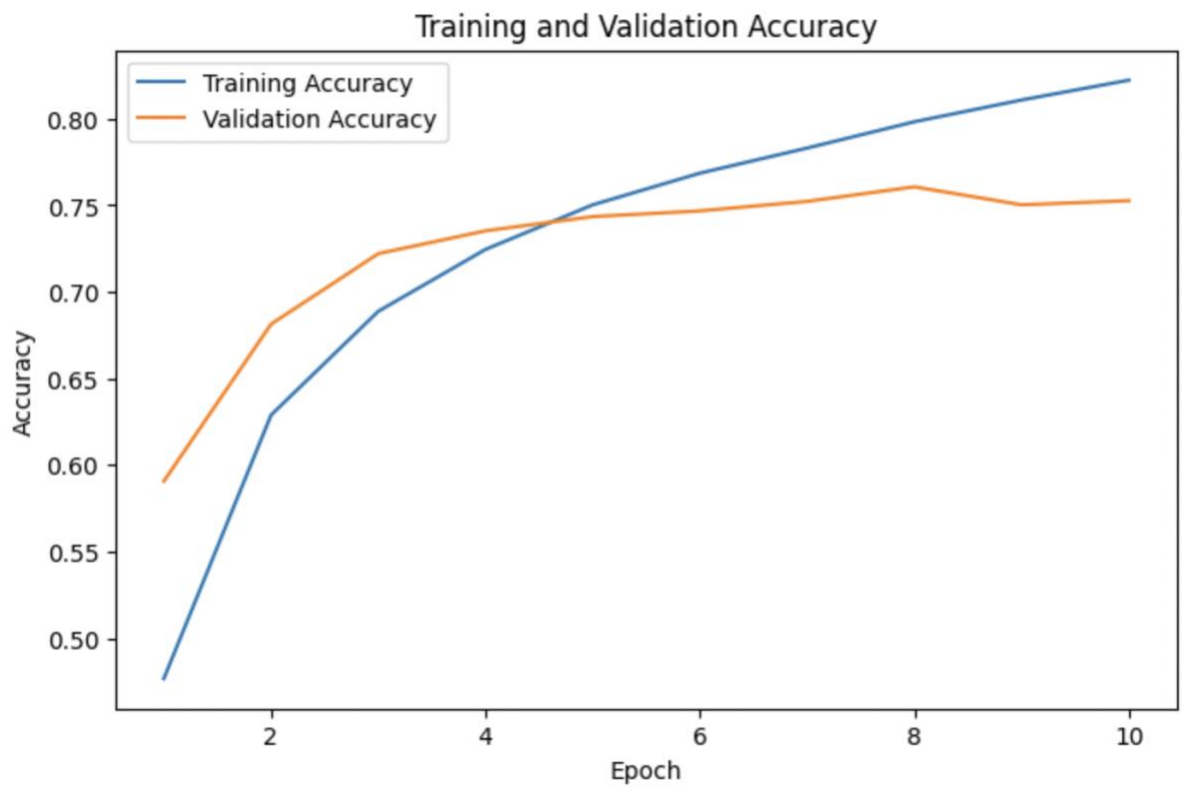
The loss function used for our model was the Cross Entropy Loss function. Optimizers that were tested include SGD and Adam, but the Adom optimizer was ultimately chosen due to its faster convergence to a solution. Training was performed for the following epoch values of 5, 10, and 15.

- **Results:** The final results for the model were a test accuracy of 75% and a training time of approximately 330 seconds.
- **Key Findings:** Initial model test accuracy contained 2 convolutional layers and was 68%. This was because the model did not utilize as many epochs as necessary to maximize model performance. Increasing the number of epochs from 5 to 10, and then from 10 to 15 further increased model accuracy from 68% in the start of our testing to 71%. Finally, increasing the capacity of the model by adding another convolutional layer increased the accuracy of the model from 71% to 73%. We needed to decrease the number of epochs from 15 to 10 to minimize overfitting from adding an additional convolution layer. In doing so, model accuracy peaked with a test accuracy of 75%, effectively surpassing the goal of 70% test accuracy. Therefor, accuracy increased as model depth increased.

Adding a third convolutional layer substantially improved performance, and the best performance occurred around 10 epochs.

- **The Cause of Overfitting:** Training accuracy always continued to increase after each subsequent epoch, regardless of the total number of epochs used. However, with validation accuracy, this was not the case. Validation accuracy plateaued and even decreased somewhat during our analysis of the total epochs. When there is a large gap between training accuracy and validation accuracy, and validation accuracy decreases, this is a major indication of overfitting. Below are the graphs from our models plotting Training and Validation Accuracy and Loss which illustrate this concept.






The total number of parameters used in the model was 620,362 and is shown in the code below.

```
total_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print("Total trainable parameters:", total_params)
```

```
Total trainable parameters: 620362
```

- **What Helped Improve Performance:** The main things that helped improve performance were adding additional convolutional layers, using the Adam optimizer, applying dropout, and using the proper number of epochs.

  Increasing the total number of convolutional layers from two to three dramatically raised the model's capacity to recognize more complex and abstract features from the images. With only two layers, only the most simple features could be understood by the model such as edges and textures, and upgrading to deeper layers heightened the model's ability to differentiate between actual shapes and entire objects. With only increasing the number of convolutional layers from two to three and using a total of 128 filters for the third convolutional layer, the model was able to comprehend multidimensional feature representations and ultimately test accuracy increased a total of 7% from 68% to 75%.

  The Adam optimizer was better suited for the task in that it was faster and more stable compared to Stochastic Gradient Descent. The result is a more efficient model in less time and reduces the total number of epochs necessary.

  Moreover, dropout was applied to reduce overfitting by randomly deactivating 25% of neurons during training. This allowed the model to become more robust in that it could concentrate on more general features instead of the noise within the model.

  Finally, the last major change which resulted in a more effective model was choosing the correct number of epochs to avoid overfitting while still maximizing performance from the model. With too few of epochs, the model resulted in underfitting with a test accuracy of 68% because the model had not had the chance to learn enough patterns from the data. Choosing the total of epochs at 10 was perfect with our third convolutional layer approach in that it avoided overfitting the data while still allowing the model to capitalize on the existing information and patterns.

- **What Did Not Help Performance:** Select choices negatively impacted the model. One of these was using too many epochs. Overfitting is a real concern when training convolutional neural networks or any neural network for that matter. This occurs when training accuracy continued to increase while the validation loss began to rise or stay stead. With overtraining, the model actually memorizes the training data instead of learning from it and being able to apply the data directly instead of focusing on the noise. In our case, after 10 epochs, validation performance stopped improving dramatically which signified that we were approaching overfitting the model. Moreover, simple models resulted in a lower accuracy, but having too many layers can result in overfitting the model as well. Staying at two convolutional layers had an accuracy within the range of 68%-71% while the full potential of the model was realized with adding a third

convolutional layer and having the test accuracy climb to 75%. Having a higher capacity within the model plays a central role in improving the model's capability. That is the central purpose of learning convolutional neural networks and deep learning in general which is to achieve heights that a simple model would struggle to reach.

4. **Conclusion:** This assignment taught the correlation between simplistic image filtering and how a deep learning model can detect image classification on its own. Image filtering is an important foundation for how deep learning image models work and for computer vision in general. Using a deep learning model with convolutional layers as opposed to a simplistic model with less convolutional layers raises model accuracy in general and with our case, it improved model accuracy from 68% to 75%. I look forward to more opportunities to utilize and build upon this concept in the future.

**Sources Used:**


- **https://chatgpt.com/share/e/69d2f79c-2f68-800c-9347-f0d1f40c0d16**
- **Raschka, S., Liu, Y., Mirjalili, V., & Dzhulgakov, D. (2022).**

**Machine learning with PyTorch and scikit-learn: Develop machine learning and deep learning models with Python.**
**Packt Publishing.**